Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

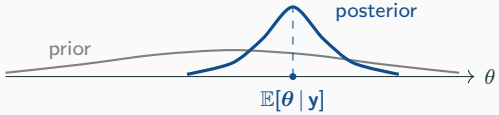
Background: neural Bayes estimators

Bayesian inference

- Fix a model: parameters $\boldsymbol{\theta}$ with prior $p(\boldsymbol{\theta})$, and data $\mathbf{y}$ with likelihood $p(\mathbf{y} \mid \boldsymbol{\theta})$.
- Seeing $\mathbf{y}$, Bayes' rule updates the prior into the **posterior distribution**:

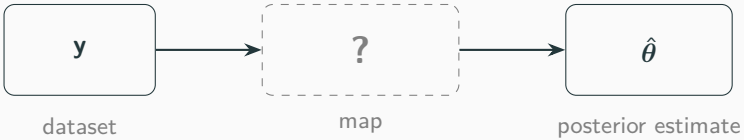
$$p(\boldsymbol{\theta} \mid \mathbf{y}) \;\propto\; p(\mathbf{y} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

- The posterior is our full, updated belief about $\boldsymbol{\theta}$.

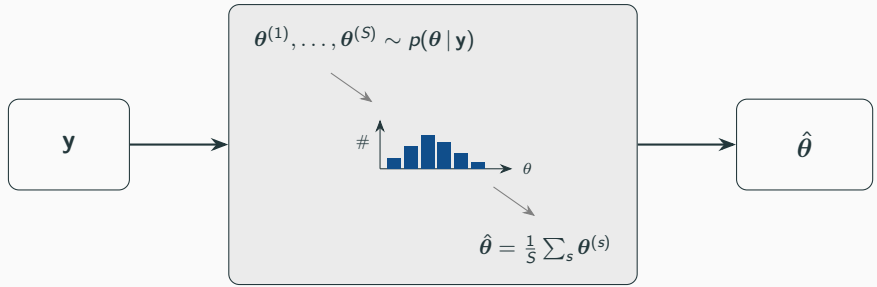


Point estimation

- Often we do not want the whole posterior — just a single **point estimate** $\hat{\boldsymbol{\theta}}$ of $\boldsymbol{\theta}$.
- The usual choice is the **posterior mean** $\hat{\boldsymbol{\theta}} = \mathbb{E}[\boldsymbol{\theta} \mid \mathbf{y}]$.
- But it is rarely available in closed form — so we need a **procedure** that turns $\mathbf{y}$ into $\hat{\boldsymbol{\theta}}$:

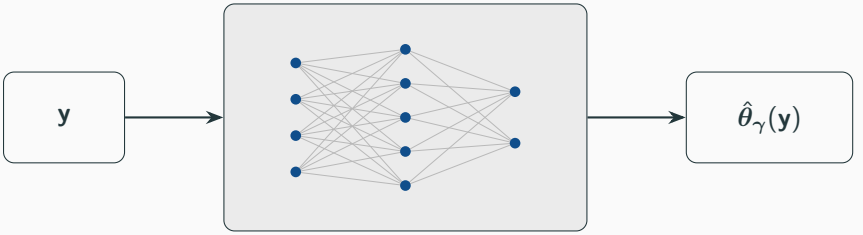


Filling the box (1): Markov chain Monte Carlo



- Accurate and general — but must be re-run from scratch for every new **y**, and can be slow.
- Do inference for *many* datasets and you pay that cost *every time*.

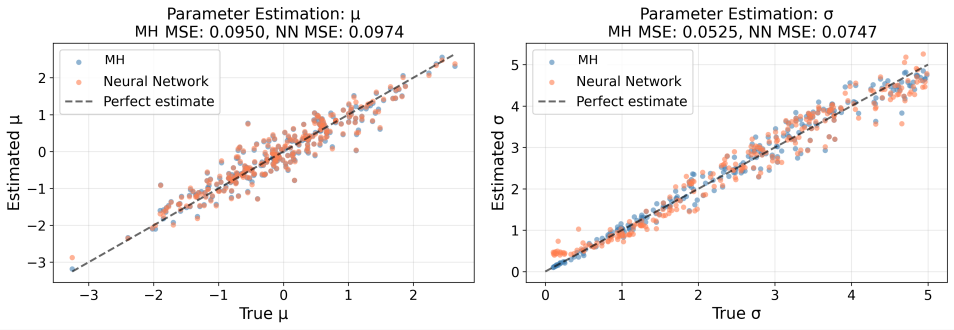
Filling the box (2): a neural network



- A **neural network** $\hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y})$, trained *once* — a *neural Bayes estimator*.
- Training data is free: **simulate** pairs $(\boldsymbol{\theta}, \mathbf{y})$ and adjust $\boldsymbol{\gamma}$ so $\hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}) \approx \boldsymbol{\theta}$.
- Then each new dataset is a single **forward pass** — the cost is amortised.

Does it work?

A simple test: estimate the mean μ and scale σ of a normal model — network vs. MCMC, on held-out data.



The network (orange) matches MCMC (blue): it has learned the posterior-mean map. *Why*? — the loss it was trained with.

The loss picks which summary you estimate

The network minimises the **Bayes risk** $L(\gamma) = \mathbb{E}_{\boldsymbol{\theta},\mathbf{y}}\big[\ell\big(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y})\big)\big]$ — and the loss ℓ decides *which* posterior summary it targets:

loss $\ell(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})$	best estimate (its minimiser)
squared $\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|^2$	posterior **mean** $\mathbb{E}[\boldsymbol{\theta} \mid \mathbf{y}]$
absolute $\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|$	posterior **median**
quantile $(\alpha - \mathbf{1}_{\{\boldsymbol{\theta} < \hat{\boldsymbol{\theta}}\}})(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$	posterior **quantile**

Squared error is exact: $\arg\min_f \mathbb{E}\big[\|\boldsymbol{\theta} - f(\mathbf{y})\|^2\big] = \mathbb{E}[\boldsymbol{\theta} \mid \mathbf{y}]$. We train with squared error — so the network targets the posterior mean, exactly what it matched on the last slide.

Our contribution: variance reduction

The observation this thesis starts from

We never have the true risk — we estimate it by **Monte Carlo** over simulated draws:

$$L(\gamma) \;\approx\; \frac{1}{N}\sum_{i=1}^{N} \left\|\boldsymbol{\theta}_i - \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}_i)\right\|^2, \qquad (\boldsymbol{\theta}_i, \mathbf{y}_i) \text{ simulated.}$$

- So the loss — and the **gradient** that drives training — is a random quantity with variance.
- That variance comes from *how we sample the loss*, not from the inference problem.
- So we can reduce it, with the classical variance-reduction tools of Monte Carlo.

Idea 1: Rao–Blackwellisation

A classical trick for killing Monte Carlo noise.

- Estimating an average by sampling? If part of it can be computed **exactly**, do that — replace the sampled piece by its exact **conditional mean**.
- Conditioning-and-averaging can only *remove* variance, never add it.

noisy draws $\longrightarrow$ replace one block by its exact mean $\longrightarrow$ tighter estimate

Named after the Rao–Blackwell theorem.

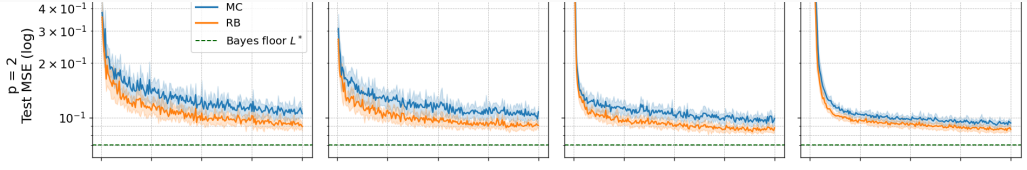
Rao–Blackwellising the loss — and why it must help

Split $\boldsymbol{\theta} = (\tau, \boldsymbol{\beta})$ with $\boldsymbol{\beta}$ conditionally conjugate; integrate $\boldsymbol{\beta}$ out by replacing its sampled target with $\mathbb{E}[\boldsymbol{\beta} \mid \tau, \mathbf{y}]$. For the gradient g,

$$\mathrm{Var}(g) \;=\; \underbrace{\mathrm{Var}\left(\mathbb{E}[g \mid \tau, \mathbf{y}]\right)}_{\mathrm{Var}(g_{\mathrm{RB}})} \;+\; \underbrace{\mathbb{E}\left[\,\mathrm{Var}(g \mid \tau, \mathbf{y})\right]}_{\succeq 0}.$$

- Law of total variance $\Rightarrow\ \mathrm{Var}(g_{\mathrm{RB}}) \preceq \mathrm{Var}(g)$.
- Never increases gradient variance — any model, any architecture.
- Linear net: the reduction reaches the fitted weights too.

Does it pay off? Bayesian linear regression



MC (blue) vs RB (orange) vs Bayes floor L^* (green) — across batch sizes 4, 16, 64, 256.

lower loss in **every** grid cell

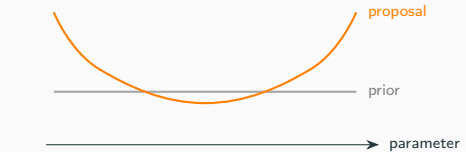
closes $\sim$**half** (52%) of the gap to L^*

matches the standard NBE at $\frac{1}{4}$ **the batch size**

Idea 2: importance sampling

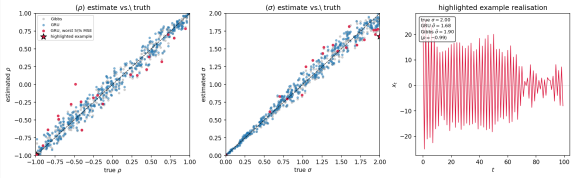
A more general lever — **no conjugacy required**.

- Simulate from a **proposal** that oversamples the informative regions, then **reweight** back to the original risk (unbiased for any proposal).
- But no guarantee: bad weights can *increase* variance and shrink the effective sample size.

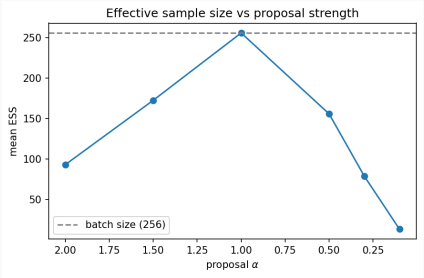


a proposal oversampling the hard edges, then reweighted

Importance sampling: the result



On an AR(1) model, the GRU already tracks the Gibbs (Bayes) reference across the prior — **little error left to recover**.



Push the proposal hard and the effective sample size **collapses** — only adding noise.

General, yes — but here reweighting found no error worth recovering. No guarantee, and it did not help.

Conclusion

Conclusion

Training a neural Bayes estimator is itself Monte Carlo — so its training variance can be reduced.

- **Rao–Blackwellisation** (needs a conjugate block): a *provable* reduction — closes ∼half the gap to the Bayes floor and matches the standard estimator at $\frac{1}{4}$ the batch size.
- **Importance sampling** (needs nothing special): more general, but unguaranteed and problem-dependent — here it did not help.

Conjugacy — the classical trick behind Gibbs sampling — is just as useful for *training* the estimators meant to replace it.

Thank you